

7.微服务检验加论文

完整流程：一边验证跨服务调用 + 一边复现TraceVAE论文

整体分**5大阶段**，同时达成两个目标：

1. Jaeger可视化看到 `frontend → user-extra-svc` 真实跨服务调用（作业联调实锤）
2. 采集真实Trace数据集 → 格式转换 → 喂给TraceVAE模型做无监督异常检测复现

整体架构

代码块

- 1 `frontend(Java) → HTTP → user-extra-svc(Python Flask自研积分服务)`
- 2 `↓OTLP埋点` `↓OTLP埋点`
- 3 `OpenTelemetry采集 → Jaeger存储&UI可视化 → 导出TraceJSON数据集 → TraceVAE图VAE训练/异常检测`

阶段1：K8s部署Jaeger All-in-One（云服务器直接执行）

1.1 创建命名空间

代码块

- 1 `kubectl create ns observability`

1.2 部署Jaeger官方all-in-one（支持OTLP接收OpenTelemetry数据）

代码块

- 1 `kubectl apply -f - <<EOF`
- 2 `apiVersion: apps/v1`
- 3 `kind: Deployment`
- 4 `metadata:`
- 5 `name: jaeger-all-in-one`
- 6 `namespace: observability`
- 7 `spec:`
- 8 `replicas: 1`

```
9     selector:
10       matchLabels:
11         app: jaeger
12     template:
13       metadata:
14         labels:
15           app: jaeger
16       spec:
17         containers:
18           - name: jaeger
19             image: jaegertracing/all-in-one:1.58
20             env:
21               - name: COLLECTOR_OTLP_ENABLED
22                 value: "true"
23             ports:
24               - containerPort: 16686 # UI页面
25               - containerPort: 4317 # OTLP gRPC接收埋点数据
26     ---
27     apiVersion: v1
28     kind: Service
29     metadata:
30       name: jaeger-all-in-one
31       namespace: observability
32     spec:
33       selector:
34         app: jaeger
35       ports:
36         - port: 16686
37           targetPort: 16686
38           name: ui
39         - port: 4317
40           targetPort: 4317
41           name: otlp
42 EOF
```

1.3 确认启动正常

代码块

```
1 kubectl get pods -n observability
2 kubectl get svc -n observability
```

Jaeger访问地址：后续用 `jaeger-all-in-one.observability.svc.cluster.local:4317` 作为埋点上报地址

阶段2：给自研user-extra-svc（Flask Python）添加OpenTelemetry埋点（关键）

2.1 修改你本地Windows里的 `app.py`（user-extra-svc主程序）

先安装埋点依赖，写入Flask追踪代码：

代码块

```
1  from flask import Flask
2  import os
3  # OpenTelemetry全量埋点
4  from opentelemetry import trace
5  from opentelemetry.sdk.trace import TracerProvider
6  from opentelemetry.sdk.trace.export import BatchSpanProcessor
7  from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
   OTLPSpanExporter
8  from opentelemetry.instrumentation.flask import FlaskInstrumentor
9  from opentelemetry.instrumentation.requests import RequestsInstrumentor
10
11 # 初始化追踪
12 trace.set_tracer_provider(TracerProvider())
13 otlp_exporter = OTLPSpanExporter(
14     endpoint=os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT","jaeger-all-in-
   one.observability.svc.cluster.local:4317")
15 )
16 trace.get_tracer_provider().add_span_processor(BatchSpanProcessor(otlp_exporter
   ))
17
18 app = Flask(__name__)
19 # 自动埋点Flask所有接口
20 FlaskInstrumentor().instrument_app(app)
21 # 自动埋点requests跨服务请求
22 RequestsInstrumentor().instrument()
23
24 # 你原来的业务代码不变
25 user_data = {
26     "10086": {"level": "VIP", "credit": 888, "register_days": 365},
27     "10001": {"level": "普通", "credit": 200, "register_days": 90},
28     "10002": {"level": "VIP", "credit": 1200, "register_days": 500}
29 }
30
31 @app.route("/health")
32 def health():
33     return {"status": "ok"}
```

```

34
35 @app.route("/api/user/info/<uid>")
36 def query_user(uid):
37     u = user_data.get(uid,{"level":"未知","credit":0,"register_days":0})
38     return {
39         "service":"user-extra-svc",
40         "user_id":uid,
41         "level":u["level"],
42         "credit":u["credit"],
43         "register_days":u["register_days"]
44     }
45
46 @app.route("/api/user/credit/deduct/<uid>/<num>")
47 def deduct(uid,num):
48     num = int(num)
49     u = user_data[uid]
50     u["credit"] -= num
51     return {
52         "service":"user-extra-svc",
53         "user_id":uid,
54         "deduct_num":num,
55         "remaining_credit":u["credit"],
56         "status":"扣除成功"
57     }
58
59 if __name__ == "__main__":
60     app.run(host="0.0.0.0",port=8080)

```

2.2 修改Dockerfile，安装埋点依赖

替换你本地 `user-extra-svc/Dockerfile` 内容：

代码块

```

1 FROM python:3.9-slim
2 WORKDIR /app
3 COPY . .
4 # 安装flask+otel全套埋点
5 RUN pip install flask requests opentelemetry-api opentelemetry-sdk
    opentelemetry-instrumentation-flask opentelemetry-instrumentation-requests
    opentelemetry-exporter-otlp-proto-grpc -i
    https://mirrors.aliyun.com/pypi/simple --trusted-host mirrors.aliyun.com
6 ENV OTEL_EXPORTER_OTLP_ENDPOINT=jaeger-all-in-
    one.observability.svc.cluster.local:4317
7 ENV OTEL_SERVICE_NAME=user-extra-svc

```

```
8 EXPOSE 8080
9 CMD ["python", "app.py"]
```

2.3 Windows重新构建镜像、打包tar、上传云服务器

代码块

```
1 # Windows PowerShell
2 cd D:\gcr-microservice-tar\user-extra-svc
3 docker build -t user-extra-svc:v1 .
4 docker save -o user-extra-svc.tar user-extra-svc:v1
```

上传 `user-extra-svc.tar` 到云服务器 `/opt/microservice-project/`

2.4 云服务器重新导入镜像、重启部署

代码块

```
1 cd /opt/microservice-project
2 eval $(minikube docker-env)
3 docker load -i user-extra-svc.tar
4 kubectl apply -f microservices-demo/kubernetes-manifests/user-extra-svc.yaml
```

阶段3：前端frontend开启调用（自带埋点，自动上报Trace）

microservices-demo官方Java微服务已经内置OpenTelemetry埋点逻辑，只需保证环境变量正确：

1. 已经配置 `USER_EXTRA_SVC_ADDR=user-extra-svc:8080`
2. Java容器内自动识别集群DNS，发起HTTP调用时自动生成父Span，user-extra-svc生成子Span，整条调用链会拼接在一起

验证跨服务链路（作业可视化实锤）

1. 端口转发打开Jaeger UI

代码块

```
1 kubectl port-forward svc/jaeger-all-in-one -n observability 16686:16686
```

2. 本地浏览器访问 `http://47.94.7.122:16686`
3. 操作前端页面下单/访问会员接口，触发 `frontend → user-extra-svc` 请求
4. Jaeger查询框选择service=frontend，点开一条Trace：
 - 第一层Span: `frontend`
 - 第二层子Span: `user-extra-svc/api/user/info/10086`

✅ 这张截图就是跨微服务调用成功最有力证明

阶段4：导出真实Trace数据集（供给TraceVAE论文复现）

4.1 导出Jaeger Trace为JSON格式数据集

两种导出方式：

1. API批量导出（适合采集大批量数据）

代码块

```
1 # 示例：导出最近1小时所有Trace到traces.json
2 curl "http://localhost:16686/api/traces?limit=10000&start=$(date -d '1 hour ago' +%s)000000&end=$(date +%s)000000" -o traces_raw.json
```

2. 小批量手动复制单条Trace JSON用于调试

4.2 按照TraceVAE源码要求做数据预处理

TraceVAE仓库：<https://github.com/NetManAIOps/TraceVAE>

1. 克隆代码、安装依赖

代码块

```
1 git clone https://github.com/NetManAIOps/TraceVAE.git
2 cd TraceVAE
3 pip3 install -r requirements.txt
```

2. 把导出的 `traces_raw.json` 按照源码 `sample_dataset` 结构整理格式（调用图结构：节点=服务，边=调用关系，特征=延迟、返回码、服务名）

3. 执行官方预处理脚本

代码块

```
1 python3 -m tracegnn.cli.data_process preprocess -i ./raw_traces_folder -o
  ./processed_dataset
```

预处理后生成模型可读取的图结构张量数据集

阶段5：TraceVAE模型训练 + 异常检测复现

5.1 正常样本训练（先用健康Trace拟合模型）

代码块

```
1 bash train.sh ./processed_dataset
```

训练完成权重保存在 `results/train/models/final.pt`

5.2 注入异常场景、测试模型检测能力（论文核心实验）

制造异常用来验证无监督检测效果：

1. 人为修改user-extra-svc代码，模拟**延迟飙升、接口报错、调用链路断裂**；
2. 采集异常Trace，同样预处理；
3. 运行评估脚本

代码块

```
1 bash test.sh results/train/models/final.pt ./processed_dataset
```

模型会计算每条Trace的**重构误差**，误差超过阈值判定为异常，完全复现论文《Unsupervised Anomaly Detection on Microservice Traces through Graph VAE》无监督图VAE检测逻辑。

关键区分：哪些是作业、哪些是论文

1) 微服务作业得分项（全部落地）

1. 自研user-extra-svc积分微服务（3个业务接口）
2. Windows离线镜像构建、Minikube K8s部署
3. frontend配置跨服务调用地址

4. Jaeger界面直观截图 `frontend → user-extra-svc` 调用链，证明跨服务通信成功

2) TraceVAE论文复现拓展项

1. OpenTelemetry标准化采集真实生产级Trace（替代论文仿真样例数据集）
2. 数据格式对齐、预处理
3. 图VAE模型训练、正常/异常Trace对比检测实验
4. 可写入论文**数据集构建、实验环境、结果分析**章节

快速兜底校验命令

1. 确认user-extra-svc埋点上报正常

代码块

```
1 kubectl logs deploy/user-extra-svc
```

无OTLP连接报错=上报Jaeger正常

2. Busybox验证集群网络

代码块

```
1 kubectl run test-box --image=busybox:1.36 --rm -it -- wget -qO- http://user-extra-svc:8080/health
```

返回 `{"status": "ok"}` =集群网络完全通畅

3. Jaeger UI能看到两条服务的Span=跨服务调用链路成型

离线方案：Windows本地拉取Jaeger镜像→打包tar→上传阿里云→导入Minikube

步骤1：Windows本地PowerShell操作（你电脑上执行）

1. Windows本地Docker拉取镜像（你本地网络能连Docker Hub）

代码块

```
1 docker pull jaegertracing/all-in-one:1.45
```

2. 导出镜像为tar包

代码块

```
1 docker save -o jaeger-1.45.tar jaegertracing/all-in-one:1.45
```

3. 把 `jaeger-1.45.tar` 通过Xftp/rz上传到阿里云服务器 `/opt/microservice-project/` 目录

步骤2：阿里云服务器操作，把tar传入Minikube虚拟机内部Docker

2.1 先把服务器上的tar拷贝进Minikube节点

代码块

```
1 # 将服务器里的jaeger-1.45.tar复制到minikube虚拟机/tmp目录
2 minikube cp /opt/microservice-project/jaeger-1.45.tar /tmp/jaeger-1.45.tar
```

2.2 进入minikube虚拟机，加载镜像

代码块

```
1 minikube ssh
2 # 虚拟机内加载tar
3 docker load -i /tmp/jaeger-1.45.tar
4 # 验证镜像存在
5 docker images | grep jaegertracing/all-in-one
6 exit
```

2.3 清理旧失败部署，重新部署Jaeger

代码块

```
1 # 删除之前异常资源
2 kubectl delete deploy jaeger-all-in-one -n observability
3 kubectl delete svc jaeger-all-in-one -n observability
4
5 # 重新部署yaml (镜像1.45)
6 kubectl apply -f - <<EOF
7 apiVersion: apps/v1
```

```
8  kind: Deployment
9  metadata:
10   name: jaeger-all-in-one
11   namespace: observability
12  spec:
13   replicas: 1
14   selector:
15     matchLabels:
16       app: jaeger
17   template:
18     metadata:
19       labels:
20         app: jaeger
21     spec:
22       containers:
23       - name: jaeger
24         image: jaegertracing/all-in-one:1.45
25         imagePullPolicy: Never # 关键! 强制不从外网拉, 只用本地已加载镜像
26         env:
27         - name: COLLECTOR_OTLP_ENABLED
28           value: "true"
29         ports:
30         - containerPort: 16686
31         - containerPort: 4317
32   ---
33  apiVersion: v1
34  kind: Service
35  metadata:
36   name: jaeger-all-in-one
37   namespace: observability
38  spec:
39   selector:
40     app: jaeger
41   ports:
42   - port: 16686
43     targetPort: 16686
44     name: ui
45   - port: 4317
46     targetPort: 4317
47     name: otlp
48  EOF
```

新增 `imagePullPolicy: Never` 是核心, 防止k8s又尝试去外网拉镜像

2.4 查看启动状态

代码块

```
1 kubectl get pods -n observability
```

此时Pod会直接使用虚拟机内已经导入的镜像，状态很快变成 `1/1 Running`

万一Windows拉镜像也慢的兜底方案（先保作业得分，Jaeger延后）

你**第三档微服务作业的核心评分项已经全部完成**，Jaeger只是可视化加分项：

1. ☒ user-extra-svc三个业务接口集群内网可正常访问，返回JSON
2. ☒ frontend已配置环境变量 `USER_EXTRA_SVC_ADDR=user-extra-svc:8080`
3. ☒ K8s同命名空间Pod DNS、网络完全互通，前端业务代码100%可以调用积分服务
4. ☒ 自研微服务离线镜像构建、Minikube部署稳定Running

临时佐证跨服务调用（不用Jaeger）

1. 开终端实时看积分服务日志

代码块

```
1 kubectl logs -f deploy/user-extra-svc
```

2. 打开电商前端页面操作会员相关功能，日志里出现接口访问记录=跨服务调用真实发生，截图一样能交作业
3. 等之后网络环境好的时候，再补Jaeger埋点、TraceVAE数据集采集与模型复现实验即可

选定统一执行版本（完整5阶段OTLP标准方案，适配TraceVAE论文复现+作业联调验证）

放弃第一段旧Jaeger Agent 6831端口方案，全程用第二段OTLP 4317标准采集链路，兼容性更好、贴合TraceVAE工业数据采集标准，一步兼顾作业和论文。

阶段1：云服务器部署OTLP版Jaeger All-in-One（当前路径 `/opt/microservice-project` 直接复制）

代码块

```
1  # 1. 创建追踪专属命名空间
2  kubectl create ns observability
3
4  # 2. 完整部署支持OTLP的Jaeger
5  kubectl apply -f - <<EOF
6  apiVersion: apps/v1
7  kind: Deployment
8  metadata:
9    name: jaeger-all-in-one
10   namespace: observability
11 spec:
12   replicas: 1
13   selector:
14     matchLabels:
15       app: jaeger
16   template:
17     metadata:
18       labels:
19         app: jaeger
20     spec:
21       containers:
22         - name: jaeger
23           image: jaegertracing/all-in-one:1.58
24           env:
25             - name: COLLECTOR_OTLP_ENABLED
26               value: "true"
27           ports:
28             - containerPort: 16686
29             - containerPort: 4317
30   ---
31   apiVersion: v1
32   kind: Service
33   metadata:
34     name: jaeger-all-in-one
35     namespace: observability
36   spec:
37     selector:
38       app: jaeger
39     ports:
40       - port: 16686
41         targetPort: 16686
42     name: ui
```

```
43     - port: 4317
44       targetPort: 4317
45       name: otlp
46 EOF
47
48 # 等待启动
49 sleep 20
50
51 # 校验状态
52 kubectl get pods -n observability
53 kubectl get svc -n observability
```

确认pod状态为 `Running` 再往下操作。

阶段2：Windows本地改造user-extra-svc埋点代码与镜像

2.1 app.py 完整替换（路径 `D:\gcr-microservice-tar\user-extra-svc\app.py`）

代码块

```
1  from flask import Flask
2  import os
3  from opentelemetry import trace
4  from opentelemetry.sdk.trace import TracerProvider
5  from opentelemetry.sdk.trace.export import BatchSpanProcessor
6  from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
   OTLPSpanExporter
7  from opentelemetry.instrumentation.flask import FlaskInstrumentor
8  from opentelemetry.instrumentation.requests import RequestsInstrumentor
9
10 # OTLP上报Jaeger
11 trace.set_tracer_provider(TracerProvider())
12 otlp_exporter = OTLPSpanExporter(
13     endpoint=os.getenv("OTEL_EXPORTER_OTLP_ENDPOINT", "jaeger-all-in-
   one.observability.svc.cluster.local:4317")
14 )
15 trace.get_tracer_provider().add_span_processor(BatchSpanProcessor(otlp_exporter
   ))
16
17 app = Flask(__name__)
18 FlaskInstrumentor().instrument_app(app)
19 RequestsInstrumentor().instrument()
20
```

```

21  user_data = {
22      "10086": {"level": "VIP", "credit": 888, "register_days": 365},
23      "10001": {"level": "普通", "credit": 200, "register_days": 90},
24      "10002": {"level": "VIP", "credit": 1200, "register_days": 500}
25  }
26
27  @app.route("/health")
28  def health():
29      return {"status": "ok"}
30
31  @app.route("/api/user/info/<uid>")
32  def query_user(uid):
33      u = user_data.get(uid, {"level": "未知", "credit": 0, "register_days": 0})
34      return {
35          "service": "user-extra-svc",
36          "user_id": uid,
37          "level": u["level"],
38          "credit": u["credit"],
39          "register_days": u["register_days"]
40      }
41
42  @app.route("/api/user/credit/deduct/<uid>/<num>")
43  def deduct(uid, num):
44      num = int(num)
45      u = user_data[uid]
46      u["credit"] -= num
47      return {
48          "service": "user-extra-svc",
49          "user_id": uid,
50          "deduct_num": num,
51          "remaining_credit": u["credit"],
52          "status": "扣除成功"
53      }
54
55  if __name__ == "__main__":
56      app.run(host="0.0.0.0", port=8080)

```

2.2 同目录Dockerfile完整替换

代码块

```

1  FROM python:3.9-slim
2  WORKDIR /app
3  COPY . .

```

```
4 RUN pip install flask requests opentelemetry-api opentelemetry-sdk
   opentelemetry-instrumentation-flask opentelemetry-instrumentation-requests
   opentelemetry-exporter-otlp-proto-grpc -i
   https://mirrors.aliyun.com/pypi/simple --trusted-host mirrors.aliyun.com
5 ENV OTEL_EXPORTER_OTLP_ENDPOINT=jaeger-all-in-
   one.observability.svc.cluster.local:4317
6 ENV OTEL_SERVICE_NAME=user-extra-svc
7 EXPOSE 8080
8 CMD ["python", "app.py"]
```

2.3 Windows PowerShell打包镜像

代码块

```
1 cd D:\gcr-microservice-tar\user-extra-svc
2 docker build -t user-extra-svc:v1 .
3 docker save -o user-extra-svc.tar user-extra-svc:v1
```

上传 `user-extra-svc.tar` 到云服务器 `/opt/microservice-project/`

2.4 云服务器加载镜像、重启部署

代码块

```
1 cd /opt/microservice-project
2 eval $(minikube docker-env)
3 docker load -i user-extra-svc.tar
4 kubectl apply -f ./microservices-demo/kubernetes-manifests/user-extra-svc.yaml
```

等待user-extra-svc Pod变为 `1/1 Running`。

`kubectl get pods -n observability`

`kubectl get svc -n observability`

```
The image user-extra-svc:v1 already exists, renaming the old one with ID sha256:0052790d050e75a13cc0f0a0e
a2059050aed6f904daae6914 to empty string
Loaded image: user-extra-svc:v1
root@iZ2zeh9xais6n11u8slj4cZ:/opt/microservice-project# kubectl apply -f ./microservices-demo/kubernetes-manifests/extra-svc.yaml
deployment.apps/user-extra-svc unchanged
service/user-extra-svc unchanged
root@iZ2zeh9xais6n11u8slj4cZ:/opt/microservice-project# kubectl get pods -n observability
kubectl get svc -n observability
NAME                                READY    STATUS    RESTARTS    AGE
jaeger-all-in-one-865cbcd6df-xkdbh 1/1      Running   0           119m
NAME                                TYPE      CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
jaeger-all-in-one                  ClusterIP  10.111.106.85 <none>         16686/TCP,4317/TCP 119m
root@iZ2zeh9xais6n11u8slj4cZ:/opt/microservice-project#
```

阶段3：前端frontend配置跨服务调用地址（无需额外埋点，官方Java自带OTel）

1. 编辑前端yaml

代码块

```
1 vim /opt/microservice-project/microservices-demo/kubernetes-manifests/frontend.yaml
```

在 `containers.env` 内插入：

代码块

```
1 - name: USER_EXTRA_SVC_ADDR
2   value: "user-extra-svc:8080"
```

保存退出 `ESC :wq`

2. 刷新部署

代码块

```
1 kubectl apply -f /opt/microservice-project/microservices-demo/kubernetes-manifests/frontend.yaml
```

Java原生微服务自带OpenTelemetry埋点，调用user-extra-svc时自动生成父子Trace链路。

阶段4：Jaeger UI查看跨服务调用（作业核心截图凭证）

1. 新开一个SSH终端挂端口转发（不要关闭窗口）

代码块

```
1 kubectl port-forward svc/jaeger-all-in-one -n observability 16686:16686
```

2. 本地浏览器打开: <http://47.94.7.122:16686>

端口转发绑定 0.0.0.0 (允许所有 IP 访问, 推荐)

1. 关闭当前正在运行的 port-forward 窗口 (Ctrl+C 终止)

2. 重新执行带 `--address 0.0.0.0` 参数的转发命令, 新开终端挂着, 新增一个防火墙端口

bash

运行

代码块

```
1 kubectl port-forward svc/jaeger-all-in-one -n observability 16686:16686 --  
  address 0.0.0.0
```

输出会变成:

plaintext

代码块

```
1 Forwarding from [::]:16686 -> 16686  
2 Forwarding from 0.0.0.0:16686 -> 16686
```

此时浏览器刷新 <http://47.94.7.122:16686> 就能正常打开 Jaeger 页面。

3. 访问电商前端页面, 触发会员积分接口请求;

方式 1: 公网 IP+NodePort (直接浏览器打开)

1. 执行查看端口:

bash

运行

代码块

```
1 kubectl get svc frontend-external
```

假设输出类似:

plaintext

代码块	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
2	frontend-external	NodePort	10.105.xxx.xxx	<none>	80:30xxx/TCP	xxx

浏览器地址：<http://47.94.7.122:30818>（把 30xxx 替换你查到的端口数字）

4. Jaeger页面：Service选择 `frontend` → Find Traces，点开一条记录，清晰看到：
`frontend（父Span）` → `user-extra-svc/api/user/info/10086（子Span）`

截图留存，作为**跨微服务联调成功**的作业证明。

场景 A：已经改好 frontend Java 调用代码（完整真实链路）

1. 清理旧转发

bash

运行

代码块

```
1 pkill -f kubectl
```

2. 窗口 1：转发前端页面

bash

运行

代码块

```
1 kubectl port-forward svc/frontend 8080:80 --address 0.0.0.0
```

3. 窗口 2：转发 Jaeger

bash

运行

代码块

```
1 kubectl port-forward svc/jaeger-all-in-one -n observability 16686:16686 --address 0.0.0.0
```

4. 浏览器打开 <http://47.94.7.122:8080>，点击页面里会请求积分接口的功能按钮

5. 打开 <http://47.94.7.122:16686>，Service 选 `frontend` → Find Traces，点开一条就能看到父子层级：`frontend（父）` → `user-extra-svc/api/user/info/10086（子）`

阶段5：导出Trace数据集 → TraceVAE论文完整复现

5.1 批量导出Jaeger原始Trace JSON

代码块

```
1 curl "http://127.0.0.1:16686/api/traces?limit=10000&start=$(date -d '1 hour ago' +%s)000000&end=$(date +%s)000000" -o traces_raw.json
```

5.2 拉取TraceVAE源码环境

代码块

```
1 mkdir -p /opt/TraceVAE && cd /opt/TraceVAE
2 git clone https://github.com/NetManAIops/TraceVAE.git .
3 pip3 install -r requirements.txt
```

5.3 数据预处理（适配图VAE输入格式）

整理 `traces_raw.json` 为源码标准图结构数据集，执行预处理脚本：

代码块

```
1 python3 -m tracegnn.cli.data_process preprocess -i ./raw_traces_folder -o ./processed_dataset
```

5.4 模型训练（健康Trace拟合VAE）

代码块

```
1 bash train.sh ./processed_dataset
```

权重输出路径： `results/train/models/final.pt`

5.5 构造异常、论文无监督检测实验

1. 修改user-extra-svc代码制造故障（超时、报错、逻辑异常）；
2. 采集异常Trace并同样预处理；
3. 评估检测效果（依靠重构误差区分正常/异常链路，完全复现论文核心逻辑）

代码块

```
1 bash test.sh results/train/models/final.pt ./processed_dataset
```

快速自检命令（随时排查问题）

1. 积分服务埋点上报是否正常

代码块

```
1 kubectl logs deploy/user-extra-svc
```

无OTLP连接报错=上报正常

2. 集群网络连通验证

代码块

```
1 kubectl run test-box --image=busybox:1.36 --rm -- wget -qO- http://user-extra-svc:8080/health
```

返回 `{"status": "ok"}` 代表Pod间网络、DNS全部通畅

3. Jaeger页面出现两个服务Span = 跨服务调用链路成型

任务边界区分

1. 微服务作业必得分项

自研3接口积分微服务、Windows离线容器构建、Minikube部署、frontend跨服务地址配置、Jaeger链路截图实锤调用；

2. TraceVAE论文拓展项

真实场景Trace采集替代论文仿真数据集、图结构预处理、VAE训练、正常/异常无监督检测对比实验，可写入论文实验章节。